

C++ Basics Cheat Sheet

Direct syntax templates for C++ practice

1. Program skeleton

```
#include <iostream>

int main() {
    // write program here
    return 0;
}
```

2. Compile and run

```
g++ -Wall -Wextra -std=c++17 path/to/file.cpp -o build/
executable_name
./build/executable_name
```

3. Basic types

int	whole number
double	decimal number
bool	true / false
char	one character
std::string	text

```
int count = 0;
double value = 2.5;
bool passed = true;
char label = 'A';
std::string name = "text";
```

4. Constants

```
const double fixed_value = 240.0;
double result = fixed_value*fixed_value;
```

5. Printing

```
std::cout << "label = " << variable << "\n";
std::cout << "a = " << a << ", b = " << b << "\n";
```

6. If / else

```
if (condition) {
    // code if true
} else if (other_condition) {
    // code if second condition is true
} else {
    // code otherwise
}
```

7. Operators

=	assignment
==	equality check
!=	not equal
<, >	less / greater
<=, >=	less/greater or equal
&&	and
	or
!	not

8. For loop

Index loop:

```
for (int index = 0; index < number_of_steps; index++) {
    // repeated code
}
```

Range loop over a vector:

```
for (type element : vector_name) {
    // use element
}
```

9. While loop

Condition loop:

```
while (condition) {
    // repeated code
}
```

Read values from an input stream:

```
while (input_stream >> variable) {
    // variable stores one value read from the stream
}
```

Here `input_stream` is the file/input object. `variable` stores the value read.

10. Functions

Template:

```
return_type function_name(input_type input_name) {
    // code
    return result;
}
```

Example form:

```
double function_name(double input_value) {
    return input_value*input_value;
}
```

11. Structs

Define a new grouped type:

```
struct TypeName {
    double member1;
    double member2;
};
```

Create and access:

```
TypeName object_name{value1, value2};
std::cout << object_name.member1 << "\n";
```

12. Vectors

```
#include <vector>

std::vector<double> values{1.0, 2.0, 3.0};
values.push_back(4.0);

for (double value : values) {
    std::cout << value << "\n";
}
```

Indexing:

```
values[0] // first element
values[1] // second element
```

13. Strings

```
#include <string>

std::string text_value = "some text";
std::cout << text_value << "\n";
```

14. Read file

```
#include <fstream>

std::ifstream input_file("path/to/input.txt");

if (!input_file) {
    std::cout << "Could not open input file\n";
    return 1;
}
```

Here "path/to/input.txt" is the file path. `input_file` is the object used to read it.

15. Write file

```
#include <fstream>

std::ofstream output_file("path/to/output.txt");

if (!output_file) {
    std::cout << "Could not open output file\n";
    return 1;
}

output_file << "label = " << value << "\n";
```

Here "path/to/output.txt" is the output path. `output_file` writes to it.

16. Read lines

```
#include <string>

std::string line;

while (std::getline(input_file, line)) {
    // line stores one full line from the file
}
```

17. Parse text line

```
#include <sstream>

std::stringstream line_stream(line);
std::string word;
double number = 0.0;

line_stream >> word >> number;
```

Here `line` is the full text line. `word` receives text. `number` receives a numeric value.

18. Header file

Header: `file_name.hpp`

```
#pragma once

struct TypeName {
    double member;
};

return_type function_name(TypeName input);
```

Source: `file_name.cpp`

```
#include "file_name.hpp"

return_type function_name(TypeName input) {
    // function definition
    return result;
}
```

19. CMake template

```
cmake_minimum_required(VERSION 3.16)
project(project_name)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

add_executable(executable_name
    main.cpp
    other_source.cpp
)
```

Build:

```
cmake -S . -B build           # configure project
cmake --build build           # compile project
./build/executable_name       # run executable
```

20. Safe division

```
double ratio = 0.0;

if (denominator > 0) {
    ratio = static_cast<double>(numerator) / denominator;
}
```

21. Debugging order

1. Read the first compiler error.
2. Check file name and line number.

3. Fix one thing.
4. Compile again.

22. Quick file checks

<code>pwd</code>	<i># show current folder</i>
<code>ls</code>	<i># list files here</i>
<code>cat file.cpp</code>	<i># print file content</i>
<code>grep -n "int main" file.cpp</code>	<i># find main with line</i>
<code>number</code>	
<code>git status --short</code>	<i># show changed files</i>